

OBSERVABILIDAD EN LA TOOLCHAIN



CODE



BUILD



DEPLOY



MONITOR

Sara Naredo – UO300563

Darío Formoso – UO301901



MÉTRICAS Y LOGS

Arquitectura del Software – Universidad de Oviedo

SE Radio Ep. 675 · Brian Demers & Giovanni Asproni

BRIAN DEMERS

Desarrollador en Gradle, centrado en Develocity

Experto en Java y Apache (Maybank y Shiro)

Contribuidor a OSS con tutoriales, blogs ...

Centrado en Build Observability y la reducción del ciclo de vida del software



QUE ES LA TOOLCHAIN

1

Conjunto de herramientas que transforman el código fuente en software desplegable.

2

Incluye compiladores, linters, gestores de dependencias, frameworks de testing, sistemas CI/CD, escáneres de seguridad...

3

Cada herramienta tiene entradas, salidas e interacciones con otras, es un sistema complejo.

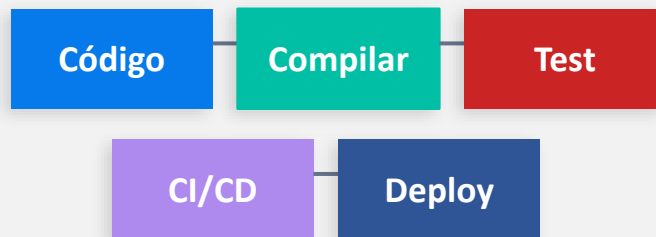
4

Un fallo en la toolchain bloquea el camino a producción igual que un fallo en producción.

QUE ES LA OBSERVABILIDAD

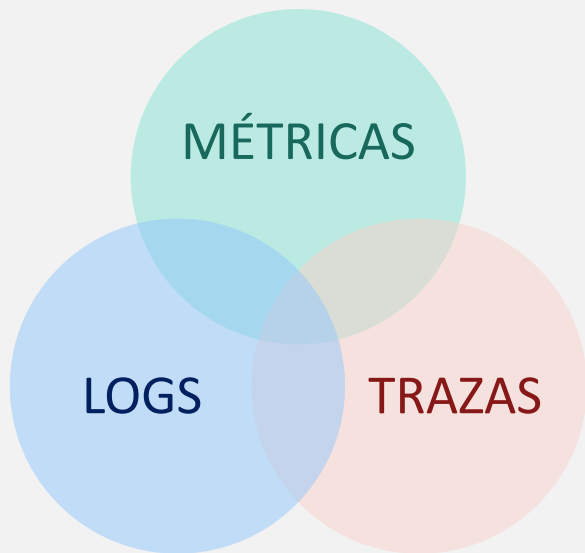
- Capacidad de entender qué está pasando realmente dentro del proceso de construcción (*build*).
- Explicación de la "caja negra"

*Proceso de desarrollo estándar
Funciona como una caja negra*



QUE ES LA OBSERVABILIDAD

Lo que hay que observar: Los 3 pilares de la observabilidad en la Toolchain



QUE ES LA OBSERVABILIDAD

Observabilidad: producción vs toolchain

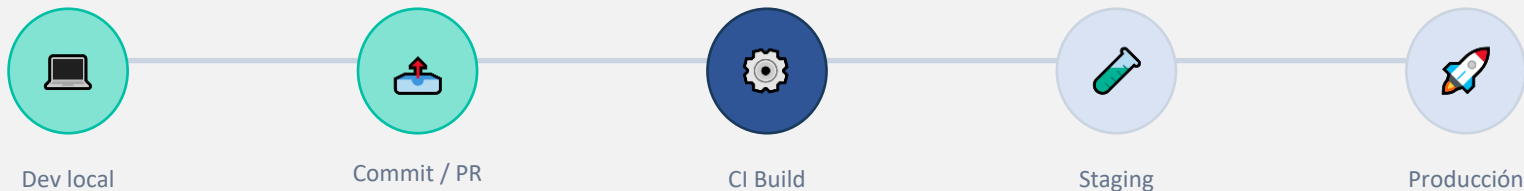
Toolchain

- ▶ Logs de compilación
- ▶ Trazas de tareas de build
- ▶ Métricas de duración
- ▶ Alertas de tests flaky
- ▶ **Aún no se suele implementar**

Producción

- ▶ Logs de aplicación
- ▶ Trazas distribuidas
- ▶ Métricas de rendimiento
- ▶ Alertas ante fallos
- ▶ **Práctica estándar**

Shift-Left de la Observabilidad



CUANTO ANTES SE DETECTE UN PROBLEMA, MEJOR

Developer Experience

Developer Productivity Engineering (DPE)

Ingeniería dedicada a reducir el tiempo de build y mejorar flujos de trabajo

Principales problema que ralentizan la productividad

TEST FLAKY

Falla de forma intermitente sin cambios de código, desperdiciando horas de debugging.

CICLO DE FEEDBACK LENTO

Si el ciclo de feedback es >10-20 min, los desarrolladores pierden el contexto y productividad (context switching).

DEPENDENCIAS

Versiones duplicadas en classpath, librerías obsoletas dan fallos en producción evitables si se detectan antes.

Pasos para mejorar la observabilidad

Actualmente, las herramientas para observabilidad son muy limitadas y poco robustas

1

Medir lo básico

- Tiempo de build local y CI
- % tests flaky
- Número de warnings

2

Instrumentar

- Activar telemetría de build tools
- Volcar datos a logs o BBDD

3

Alertar

- Tiempo máximo de build
- Umbral de % de flaky rate
- Dependencias vulnerables

Sistemas greenfield vs legacy

Proyecto Greenfield – construido desde 0

- Instrumentar todo desde el inicio, aunque no haya datos que analizar
- Tomar decisiones sobre los datos recolectados para crear alertas
- Automatizar el envío de emails, la creación y asignación de issues, ...

Sistema Legacy – ya construidos que se siguen usando

- Centrarme en los problemas concretos
- Investigar métricas como el % de tiempo que ocupan los test, si puedo obtener feedback de otra parte, ...

Ejemplo de problema con test flacky

1. Tiempo perdido

El desarrollador investiga un fallo que no es suyo. Puede costar horas.

2. Afecta a todo el equipo

Es necesario avisar a todo el equipo y gastar recursos

SOLUCIÓN APLICANDO OBSERVABILIDAD

Monitorizar el % de tests flaky y disparar alertas cuando supere un umbral

3. Hacer re-runs automáticos

Una solución puede ser hacer ejecuciones que oculten el problema

4. Cuarentena de tests

También podemos mover los tests flaky a un proyecto separado hasta que se corrijan

Como ahorrar tiempo con los warnings del compilador

EL PROBLEMA

- Con builds de 20+ min nadie lee los logs completos
- Un 'green build' enmascara cientos de warnings
 - Los warnings ignorados son bugs silenciosos en producción
- La deuda técnica de build crece sin que nadie la vea

LA SOLUCIÓN

- Tratar los warnings como errores
- Hacer trackings del número de errores y como se reduce
- Considerar una meta decreciente para llegar a los objetivos
 - Utilizar la IDE para feedback instantáneo y el CI como quality gate

Riesgos y oportunidades

OPORTUNIDADES

- ▶ Clasificación automática de fallos
- ▶ Detección de patrones en errores recurrentes
- ▶ Sugerencia de re-runs inteligentes
- ▶ Análisis predictivo de qué tests son relevantes ejecutar
- ▶ Optimización de avisos

- ▶ Más código generado causa builds más lentas
- ▶ Los tests generados por IA a veces no validan nada real
- ▶ Origen del código difícil de rastrear
- ▶ Código de terceros con vulnerabilidades no detectadas

RIESGOS

Porque invertir en observabilidad desde el punto de vista empresarial



**Acelera
solu-
ciones**

**Reduce
riesgos**

**Ahorra
costes**

**Reduce
el time-
to-
market**

FACTOR HUMANO

Cultura y organización

En algunos casos, los datos recogidos no se observan ni se utilizan

Recoger información no es útil si no se utiliza para obtener mejoras

A veces simplificar el código es la mejor forma de mejorarlo

No siempre es necesario todo el código, test o herramientas, hay que estudiar cada caso

Se necesita entender la relación entre build rápido y más productividad

Si tu build tarda 20 min y falla, ya llevas media hora sin escribir código útil. Si lo reduces a 2 min, puedes iterar 10 veces en el mismo tiempo

Es importante educar sobre que problemas puede resolver cada equipo

En equipos grandes, los fallos de CI se delegan a equipos sin contexto suficiente para resolverlos

FUTURO DE LA OBSERVABILIDAD

Prioridad a la productividad segura

IA PARA ANÁLISIS DE FALLOS

Pattern matching automático para reconocer fallos, reducir ruido y acelerar resoluciones

IMPORTANCIA DE LA SEGURIDAD

Obligar a registrar entradas y salidas del pipeline y conocer en todo momento de donde viene el código.

POLÍTICAS Y AUTMATIZACIÓN

Definir reglas y validarlas automáticamente en el pipeline.



PREGUNTAS